QA ATELIER · TEST PROJECT

RWA Test Project Documentation

Requirements · Test Case Catalog · RTM · Environment Setup · Operating Model · CTFL Key Points

Author	Tomas Xavier Santos
Project	QA Atelier (tom-xs/qa-atelier)
AUT	Cypress Real World App
Date	2026-07-30
Version	1.0

Contents

1. Introduction

1.1 Purpose of this document

This document is the single reference for the QA Atelier test project against the Cypress Real World App (RWA). It consolidates the functional requirements of the application under test, the catalog of existing and planned test cases, a requirements traceability matrix (RTM), the environment setup procedures, the project's operating model, and the conventions to follow before writing new tests. It closes with a glossary and a mapping to the ISTQB Certified Tester Foundation Level (CTFL) v4.0 syllabus, so the project doubles as exam preparation material.

1.2 Project context

QA Atelier (`tom-xs/qa-atelier`) is a personal portfolio project: a monorepo that practices professional QA workflows end to end — test planning, test design, automation across several frameworks, CI/CD, and issue-based test management on GitHub. It is built to look and operate like a real-world QA engagement, not a tutorial exercise: branch protection, required CI checks, labeled test-case issues, and traceability from requirements to automated specs.

1.3 Application under test

The **Cypress Real World App (RWA)** is an open-source payment application built by Cypress as a realistic testing target. It mimics a Venmo-style service: users sign up, link bank accounts, send payments, request money, browse transaction feeds, like and comment on transactions, and receive notifications. It runs locally with a React frontend (port 3000) and an Express REST API (port 3001), backed by a file-based JSON database (`apps/rwa/data/database.json`) that ships with a seed dataset of users and transactions.

Two API characteristics matter for test design and were verified against the running server:

- **Session-cookie authentication.** `POST /login` returns the user object and sets a `connect.sid` cookie. There is no bearer token; protected endpoints are called with the session cookie.
- **Ephemeral state.** Every `yarn start` re-seeds the database from `data/database-seed.json` . A restart is a factory reset — a property the test data strategy in Section 7 exploits.

1.4 Scope

In scope: functional testing of auth, users, bank accounts, transactions, feeds, and notifications at API and UI level; CI integration of all suites; test management artifacts (this document, test-case issues, RTM).

Out of scope: performance/load testing, security penetration testing, mobile (iOS) coverage, and the RWA's optional OAuth providers (Auth0, Okta, Cognito, Google) — the local username/password strategy is the only auth flow tested.

2. Functional Requirements

Requirements are written as testable statements with stable IDs for traceability. Priorities: **P0** = critical path (must pass before any “release”), **P1** = high (every test cycle), **P2** = normal.

ID	Requirement	Priority
REQ-AUTH-001	A user with valid credentials can log in; the server returns the user object and establishes a session cookie	P0
REQ-AUTH-002	A user with an invalid password cannot log in; the API returns 401 and the UI shows an error message	P0
REQ-AUTH-003	A logged-in user can log out; the session is invalidated and protected pages redirect to sign-in	P1
REQ-AUTH-004	A new user can sign up with username, password, first and last name	P1
REQ-AUTH-005	An active session persists across page reloads (cookie-based); no re-login needed	P1
REQ-USER-001	A logged-in user can search other users by name or username	P1
REQ-USER-002	A logged-in user can view another user’s public profile (first name, last name, avatar)	P2
REQ-USER-003	A logged-in user can update their own account settings (name, email, phone)	P2
REQ-BANK-001	A logged-in user can link a bank account with bank name, routing number, account number	P1
REQ-BANK-002	A logged-in user can view their list of linked bank accounts	P1
REQ-BANK-003	A logged-in user can delete (soft-delete) a linked bank account	P2

REQ-TX-001	A logged-in user with sufficient balance can pay another user an amount with a description; the transaction appears in feeds and balances update	P0
REQ-TX-002	A logged-in user can request money from another user; the request appears as pending for the recipient	P0
REQ-TX-003	Transaction feeds are filtered correctly: Mine, Friends, Public	P1
REQ-TX-004	The recipient of a money request can accept (creates payment) or reject it	P1
REQ-TX-005	A user can like and comment on a transaction	P2
REQ-TX-006	A payment exceeding the sender's balance is rejected with an error	P1
REQ-NOTIF-001	A user receives a notification when a payment is received or a request is created for them	P1
REQ-NOTIF-002	A user can dismiss notifications; the unread counter updates	P2

3. Test Case Catalog

Test cases are managed as GitHub Issues using the `type: test-case` label and the test-case template. IDs are assigned sequentially (TC-XXX).

3.1 Existing test cases

ID	Title	Requirement	Type	Priority	Framework	Status
TC-001	Log in with valid credentials (UI)	REQ-AUTH-001	Smoke	P0	Cypress	Retired — covered by API tests
TC-002	Create a payment transaction	REQ-TX-001	Functional	P1	Cypress	Documented (issue #6)
TC-003	Request money from another user	REQ-TX-002	Functional	P1	Cypress	Documented (issue #7)
TC-004	User cannot log in with invalid credentials	REQ-AUTH-002	Smoke / Negative	P0	Cypress	Documented (issue #8)
TC-005	RWA API auth collection + CI environment	REQ-AUTH-001, REQ-AUTH-002	API Contract	P1	Postman / Newman	Documented (issue #12)

—	API: login returns user + session cookie; 401 on bad password; cookie grants `/users`	REQ-AUTH-001, REQ-AUTH-002, REQ-AUTH-005	API Contract	P0	Vitest	Automated (`api/tests/rwa/auth.test.ts`)

3.2 Planned test cases

ID	Title	Requirement	Type	Priority	Target framework
TC-006	Log out invalidates the session	REQ-AUTH-003	Functional	P1	Cypress
TC-007	Sign up creates a new account	REQ-AUTH-004	Functional	P1	Cypress
TC-008	Session persists across reload	REQ-AUTH-005	Functional	P1	Playwright
TC-009	Link a new bank account	REQ-BANK-001	Functional	P1	Cypress
TC-010	Bank accounts list shows linked accounts	REQ-BANK-002	Functional	P1	Postman / Newman
TC-011	Feeds filter Mine / Friends / Public correctly	REQ-TX-003	Functional	P1	Cypress
TC-012	Recipient accepts a money request	REQ-TX-004	Functional	P1	Cypress
TC-013	Recipient rejects a money request	REQ-TX-004	Negative	P2	Cypress
TC-014	Payment above balance is rejected	REQ-TX-006	Negative	P1	Vitest (API)
TC-015	Like and comment on a transaction	REQ-TX-005	Functional	P2	Cypress
TC-016	Notification appears for received request	REQ-NOTIF-001	Functional	P1	Cypress
TC-017	User search returns matching users	REQ-USER-001	Functional	P2	Vitest (API)

TC-018	Update account settings persists	REQ-USER-003	Functional	P2	Playwright
TC-019	API schema contract: `/transactions` response shape	REQ-TX-001	API Contract	P1	Postman / Newman
TC-020	Response-time budget: login under 500 ms	REQ-AUTH-001	Non-functional	P2	Postman / Newman

4. Requirements Traceability Matrix (RTM)

The RTM proves that every requirement has at least one test case, and shows where coverage is automated versus manual-only. It is updated whenever a test case is added, automated, or retired.

Coverage key: **A** = automated (passing spec in CI), **D** = documented (issue written, not yet automated), **P** = planned (cataloged in this document), **—** = no coverage.

Requirement	Test cases	Coverage	Gap / next action
REQ-AUTH-001	TC-001 (retired), TC-005, Vitest auth suite	**A**	Automate TC-005 collection
REQ-AUTH-002	TC-004, TC-005, Vitest auth suite	**A**	Automate TC-004 in Cypress (UI error message)
REQ-AUTH-003	TC-006	P	Write Cypress spec
REQ-AUTH-004	TC-007	P	Write Cypress spec
REQ-AUTH-005	TC-008, Vitest auth suite (cookie reuse)	**A** (API level)	UI-level check in TC-008
REQ-USER-001	TC-017	P	Write Vitest spec for `/users/search`
REQ-USER-002	—	—	Add test case (P2)
REQ-USER-003	TC-018	P	Write Playwright spec
REQ-BANK-001	TC-009	P	Write Cypress spec
REQ-BANK-002	TC-010	P	Add to Postman collection
REQ-BANK-003	—	—	Add test case (P2)
REQ-TX-001	TC-002, TC-019	D	Automate TC-002 in Cypress
REQ-TX-002	TC-003	D	Automate TC-003 in Cypress
REQ-TX-003	TC-011	P	Write Cypress spec

REQ-TX-004	TC-012, TC-013	P	Write Cypress specs
REQ-TX-005	TC-015	P	Write Cypress spec
REQ-TX-006	TC-014	P	Write Vitest spec
REQ-NOTIF-001	TC-016	P	Write Cypress spec
REQ-NOTIF-002	—	—	Add test case (P2)

Coverage summary (2026-07-30): 18 requirements — 3 with automated coverage, 4 documented, 9 planned, 3 with no test case yet (all P2). Rule: no requirement ships to "Done" with an **—** in this matrix; P0/P1 rows must reach **A** before the project is considered complete.

5. Environment Setup

All steps assume the QA Atelier repository cloned with submodules on the NixOS machine.

5.1 Nix devShell

Every tool is provided by the project flake; nothing is installed globally.

```
git clone --recurse-submodules https://github.com/tom-xs/qa-atelier.git
cd qa-atelier
nix develop              # or: direnv allow (uses .envrc)
```

`flake.lock` is committed and pins all tool versions. The devShell exports `PLAYWRIGHT_BROWSERS_PATH`, `CYPRESS_RUN_BINARY`, `ANDROID_HOME`, and `JAVA_HOME`.

5.2 Boot the RWA

```
cd apps/rwa
yarn install --legacy-peer-deps   # first time only
yarn start                        # re-seeds the DB, serves UI :3000 and API :3001
```

Sanity check:

```
curl -s -X POST http://localhost:3001/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"Heath93","password":"s3cret"}'
# expected: {"user":{"id":"...","username":"Heath93",...}}
```

State management: `yarn start` = reset to seed; `yarn start:empty` = empty database; `yarn db:seed:dev` = re-seed while stopped.

5.3 Postman (GUI)

1. Add `postman` to the devShell packages in `flake.nix` (unfree is already allowed) and re-enter `nix develop`; or run ad hoc: `NIXPKGS_ALLOW_UNFREE=1 nix shell --impure nixpkgs#postman`.
2. Sign in with a free account (collections and environments require it; the offline lightweight client does not have them). Create a **private personal workspace**. Do **not** connect the workspace to a local folder (Native Git) — that feature stores Postman's own YAML tree and conflicts with the repo layout.
3. Optional: disable Agent Mode in Settings so the "New" button stops offering AI setup.
4. Build the `RWA Auth` collection: `POST {{baseUrl}}/login` (valid + bad password), `GET {{baseUrl}}/users`, `GET {{baseUrl}}/users/{{userId}}` — assertions per issue #12 (status, `user` object, `connect.sid` cookie via `pm.cookies.has`, response time under 500 ms, chain `userId` into the environment).
5. Create environment **CI**: `baseUrl=http://localhost:3001`, `username=Heath93`, `password` with initial value `{{RWA_PASS}}` and current value `s3cret`, empty `userId`. Only initial values are exported — the real password never leaves the machine.
6. Export: collection (v2.1) to `api/collections/rwa-auth.postman_collection.json`; environment to `api/environments/ci.postman_environment.json`. Verify with `jq` that no real password is in the exported files.

5.4 Newman (CLI runner)

Newman is a devDependency of `api/` (pinned by lockfile) and also present in the devShell. Run all collections from the repo root:

```
bash api/newman/run-all.sh
```

The script skips gracefully when no collections exist, injects `RWA_PASS` via `--env-var`, and writes JUnit reports to `api/reports/` (git-ignored, uploaded as CI artifacts).

5.5 Programmatic API tests (Vitest)

```
cd api && npm install   # first time; afterwards npm ci
npm test                # vitest run --passWithNoTests
```

Requires RWA running on port 3001. Credentials come from `RWA_USER` / `RWA_PASS` env vars with `||` fallbacks to the public seed credentials (missing CI secrets expand to empty strings, which `??` would not catch).

5.6 Other frameworks (quick reference)

Framework	Install	Run

Playwright	`cd web/playwright && npm install`	`npx playwright test`
Cypress	`cd web/cypress && npm install`	`npx cypress run` (RWA must be running)
Selenium	devShell pythonEnv	`pytest web/selenium/tests/ -v`
UIAutomator	devShell Android SDK	`cd android && ./gradlew connectedAndroidTest`

5.7 GitHub secrets and CI

Repository secrets (Settings → Secrets and variables → Actions): `RWA_USER`, `RWA_PASS`. The workflows pass them as env vars; test code falls back to the public seed credentials when they are absent, but the secrets should stay configured to exercise the intended pattern.

6. How the Project Operates

6.1 Monorepo layout

```
qa-atelier/
├── .github/            # workflows, issue templates, copilot-instructions.md
├── android/            # UIAutomator tests (Kotlin + Gradle)
├── api/                # Postman collections, environments, Newman, Vitest tests
├── apps/rwa/           # Cypress Real World App (git submodule)
├── docs/               # test plan and test-case docs
├── shared/             # reusable utilities and test data
├── web/
│   ├── cypress/        # Cypress specs + page objects
│   ├── playwright/     # Playwright specs + page objects
│   └── selenium/       # Selenium tests (Python)
├── flake.nix           # devShell — all tools, pinned
├── Makefile            # make test-* entry points
└── package.json        # root scripts via npm --prefix (no workspaces)
```

Each framework folder is self-contained with its own lockfile; installs happen per folder.

6.2 Branching model

`master` is protected: changes land only via pull request, and the required CI checks must pass. Solo-maintainer reviews are set to zero approvals. Feature branches are named `feat/<topic>`, fixes `fix/<topic>`.

6.3 Test management on GitHub

- **Test cases** are Issues created from the test-case template, labeled `type: test-case` plus framework, app, and priority labels.

- **Defects** found during testing use `type: bug-found`; exploratory session notes use `type: exploratory`.
- **Status** labels (`status: automated`, `status: manual-only`) track automation progress; the RTM in Section 4 mirrors them.
- A project board tracks the flow: Backlog → Exploration → In Scripting → In Review → Done / Blocked.

6.4 CI pipelines

Workflow	Trigger	What it does
Playwright E2E	push/PR to master, weekday cron	Multi-browser E2E, uploads HTML report
Cypress E2E	push/PR to master	Starts RWA, runs specs on Chrome + Firefox matrix, uploads screenshots on failure; skips when no specs exist
API Tests	push/PR touching `api/**` or `apps/rwa/**`	Starts RWA, waits on port 3001, runs Newman collections + Vitest suite, uploads reports
Android UIAutomator	push/PR to master	Runs connected tests on an emulator (API 33) when Kotlin tests exist

6.5 AI-assisted workflow

GitHub Copilot handles inline completion and boilerplate; Kimi Code handles reasoning tasks (coverage reviews, refactors, failure diagnosis). Conventions for both live in `.github/copilot-instructions.md`. Rule: generated test code is never merged without running it locally first.

7. Preparing for Testing and Automation

7.1 Conventions (non-negotiable from day one)

- **Page Object Model** for web frameworks, **Screen Object Model** for Android — tests never locate elements directly.
- **AAA structure** (Arrange–Act–Assert) with visible phase separation in every test.
- **Selectors**: `data-test` / `data-testid` attributes only (`cy.getBySel('...')`).
- **Test names** read as `[action] [expected result]`.
- **Explicit waiting only** — conditions, never time: `expect(locator).toBeVisible()`, `should('be.visible')`, `WebDriverWait`. No `sleep`, no `cy.wait(ms)`.
- **Tests are independent** and runnable in any order.

7.2 Test data and state strategy

Exploit the RWA's ephemeral database:

- **Default state**: the seeded dataset (Heath93 et al.) after every server start.
- **Per-test setup**: create the data a test needs via API in `beforeEach` (5–10x faster than UI flows and immune to UI changes) — the "API seeding" pattern.
- **Reset**: restart the server between manual exploratory sessions; never reset mid-suite.
- **Empty-state testing**: `yarn start:empty` for signup/onboarding scenarios.
- **RWA `/testData` endpoints**: available in dev mode for injecting specific entities when a scenario needs precise state.

7.3 Isolation rules

Each test creates what it needs and cleans up after itself; no test depends on another test's side effects; shared mutable state (like account balance) is asserted relative to a value read at test start, never as a hardcoded absolute.

7.4 Pattern adoption order

Start (now): POM, AAA, explicit waiting, test isolation. Next (20+ tests): fixture factories, API seeding, tagging (`@smoke` , `@regression`), API client reuse. Later (100+ tests): schema validation, network interception, data-driven suites, visual regression. Add a pattern only when its pain is felt.

8. Glossary

Term	Meaning in this project
AUT	Application under test — here, the RWA
CI / CD	Continuous Integration / Delivery — tests run automatically on every push and PR via GitHub Actions
Collection	A group of related Postman requests saved together and run as one suite
Contract test	API test that asserts the response shape (fields, types), catching breaking changes before the UI does
devShell	Nix shell declared in `flake.nix` providing every project tool reproducibly
Environment (Postman)	Named set of variables (`baseUrl` , credentials) swapped between local and CI runs
Flaky test	A test that passes and fails without code changes — almost always a waiting or isolation defect

Newman	Postman's CLI runner; replays collection JSON in CI without a GUI
P0 / P1 / P2	Priority classes: critical path / every cycle / normal
POM	Page Object Model — a class per page encapsulating selectors and actions
Regression suite	Tests re-run after changes to prove existing behavior still works
RTM	Requirements Traceability Matrix — maps requirements to test cases and coverage status
Seed / seeding	Loading known data into the database before tests (file copy for RWA, API calls in specs)
Smoke test	Small fast suite proving the critical path works (P0 cases)
Testware	All artifacts produced for testing: this document, cases, scripts, data, environments
Vitest	TypeScript test runner used for the programmatic API tests (describe/test/expect)

9. CTFL v4.0 Key Points — Mapped to This Project

The ISTQB CTFL exam rewards connecting syllabus concepts to real practice. Every section of this project is an example of something examinable.

9.1 Chapter 1 — Fundamentals of Testing

- **Test objectives** vary by context: here the objectives are defect detection (negative API cases), confidence in the critical path (P0 smoke), and portfolio evidence of process.
- **Seven testing principles in action**: testing shows presence of defects (the `body.token` failure proved an assumption wrong, not the app); exhaustive testing is impossible (prioritized P0/P1/P2 instead); early testing saves money (API contracts before UI automation); defect clustering (auth flows get the most negative cases); pesticide paradox (exploratory sessions via `type: exploratory` issues refresh the suite); testing is context dependent (RWA's cookie auth shaped the test design); absence-of-errors fallacy (green tests against a wrong contract are worthless — verify against the real server).
- **Test process activities**: planning (this document), analysis and design (test-case issues, RTM), implementation and execution (specs + CI), evaluating completion (RTM coverage rule), completion (archive).
- **Testware**: document, issues, collections, scripts, environments — all version-controlled alongside code.
- **Traceability**: the RTM (Section 4) links requirements to tests and supports coverage evaluation — a core exam theme.

9.2 Chapter 2 — Testing Throughout the SDLC

- **Test levels**: Vitest/Newman suites sit at component/integration level (API contracts); Cypress/Playwright specs at system level (E2E); CI ties them together per commit.

- **Test types**: functional (most cases), non-functional (TC-020 response-time budget), regression (scheduled weekday runs), smoke (P0 subset).
- **Shift-left**: API testing before UI automation; contract assertions catch breaks earlier and cheaper.
- **Maintenance testing**: every RWA version bump (submodule update) triggers full regression via path-filtered workflows.

9.3 Chapter 3 — Static Testing

- **Reviews find defects early and cheaply**: every pull request is a static-testing event — the broken `api.yml` indentation and the wrong `working-directory` key were caught by review before ever executing.
- Static analysis complements this: TypeScript compilation of spec files catches errors before runtime.

9.4 Chapter 4 — Test Analysis and Design

- **Equivalence partitioning**: login credentials (valid / wrong password / unknown user); amounts (positive / zero / negative).
- **Boundary value analysis**: payment amount at 0.01, balance exactly equal to amount, balance minus one cent (TC-014).
- **Decision tables**: transaction outcomes — sufficient balance × pay/request × accept/reject combinations.
- **State transition testing**: session states (anonymous → authenticated → logged out); transaction states (pending → accepted/rejected).
- **Experience-based techniques**: exploratory charters logged as `type: exploratory` issues; error guessing produced the empty-string-secret CI defect.
- **Black-box vs experience-based balance**: catalog cases are specification-based; session notes feed new catalog entries.

9.5 Chapter 5 — Managing the Test Activities

- **Test planning**: this document is the test plan — scope, approach, resources, schedule via the project board.
- **Risk-based prioritization**: P0/P1/P2 labels map to product risk; CI runs smoke on every PR and full regression nightly.
- **Entry and exit criteria**: entry — RWA healthy (sanity login curl), branch green; exit — all P0 automated and passing, RTM has no uncovered P0/P1 rows.
- **Monitoring and control**: CI run history, artifacts (reports, screenshots, traces), and board velocity.
- **Configuration management**: everything versioned — code, tests, environments, flake.lock pinning the toolchain.
- **Defect management**: `type: bug-found` issues with reproduction steps, severity via priority labels, lifecycle on the board.

9.6 Chapter 6 — Test Tools

- **Tool selection rationale** lives in the framework decision matrix (guide §2.1): each tool was chosen for a purpose, not habit — a K2 exam favorite ("benefits and risks of test automation").
- **Automation risks demonstrated honestly**: initial over-reliance on assumed contracts (token that did not exist) and flaky timing (RWA startup race) — both classic automation pitfalls named in the syllabus.
- **Pilot first**: the auth domain was automated end-to-end before scaling to transactions — the recommended tool-introduction approach.

10. Appendix — Quick Reference

```
# Environment
nix develop
cd apps/rwa && yarn install --legacy-peer-deps && yarn start    # reset state on every start
yarn start:empty                                                # empty database

# API testing
bash api/newman/run-all.sh        # Postman collections (skips gracefully)
cd api && npm test                # Vitest programmatic tests

# Web E2E
cd web/cypress && npx cypress run
cd web/playwright && npx playwright test
pytest web/selenium/tests/ -v

# Android
cd android && ./gradlew connectedAndroidTest

# Git workflow (master is protected)
git checkout -b feat/<topic> && git push -u origin feat/<topic>  # then open a PR
```

References: Cypress Real World App (github.com/cypress-io/cypress-realworld-app) · Postman Learning Center (learning.postman.com) · Newman (github.com/postmanlabs/newman) · Playwright docs (playwright.dev) · Cypress docs (docs.cypress.io) · ISTQB CTFL v4.0 syllabus (istqb.org) · QA Atelier repository (github.com/tom-xs/qa-atelier)